

■ Python Concurrency — Lecture 4

Multiprocessing Deep Dive

Processes | Queue | Value | IPC | Real World

#	Section	Description
1	Process API Basics	Process(), start(), join(), args — full reference
2	Threading Fails — CPU	09_process1.py — why threading gives no CPU speedup
3	Multiprocessing Wins	10_process2.py — true parallel speedup
4	The if __name__ Guard	Why it is mandatory and what happens without it
5	Thread to Process	Minimal code changes to convert threading to MP
6	Queue — IPC	11_processq.py — sharing data between processes
7	Value — Shared Counter	12_process_value.py — shared primitive with auto-lock
8	Speed Comparison	Threading vs Multiprocessing benchmark analysis
9	Real World Use Cases	Image processing, ML, data pipelines
10	Interview Q&A;	Top questions with complete model answers
11	Quick Revision	Cheatsheet, tables, golden rules

■ Section 1 — Multiprocessing Process API

Complete API Reference Table

Method / Property	Purpose	Key Notes
<code>from multiprocessing import Process</code>	Import the Process class	Built-in — no pip install needed
<code>Process(target=func)</code>	Create a process — NOT started yet	Same pattern as <code>threading.Thread</code>
<code>Process(target=func, args=(a,))</code>	Create process with arguments	<code>args</code> must be a tuple — trailing comma for single arg
<code>Process(target=func, name='P1')</code>	Named process for debugging	Optional but helps in logging
<code>.start()</code>	Spawn process on a new CPU core	Returns immediately — process runs in background
<code>.join()</code>	Wait until process terminates	Must be called after <code>start()</code>
<code>.pid</code>	Process ID assigned by OS	Useful for debugging and monitoring
<code>.name</code>	Human-readable process name	Set via <code>name=</code> at creation
<code>if __name__ == '__main__':</code>	Entry-point guard — MANDATORY	Without this: infinite process spawning on Windows/macOS
<code>Queue()</code>	Shared FIFO queue between processes	<code>from multiprocessing import Queue</code>
<code>q.put(item)</code>	Add item to queue	Blocks if queue is full
<code>q.get()</code>	Remove and return item from queue	Blocks until item available
<code>q.empty()</code>	Check if queue is empty	Returns True/False
<code>Value('i', 0)</code>	Shared integer across processes	<code>from multiprocessing import Value</code>
<code>counter.value</code>	Read or write the shared number	Always use with <code>counter.get_lock()</code>
<code>counter.get_lock()</code>	Built-in lock for the Value object	No need to create separate Lock
<code>with counter.get_lock(): ...</code>	Thread/process-safe modification of Value	Prevents race conditions automatically

■ Section 2 — Why Threading Fails for CPU Tasks (09_process1.py)

The Problem — Threading + CPU Work

This file deliberately uses **threading** for a CPU-heavy task to demonstrate its limitation. The goal is to show that no matter how many threads you create, the GIL prevents true parallel execution of Python bytecode — resulting in no speed improvement over single-threaded execution.

Full Code — 09_process1.py

```
import threading
import time

def cpu_heavy():
    print('Crunching numbers...')
    total = 0
    for i in range(10**7):    # 10 million iterations — pure CPU
        total += i           # Each iteration modifies memory
    print('Done!')

start = time.time()

# Two threads — expecting 2x speedup
threads = [
    threading.Thread(target=cpu_heavy)
    for _ in range(2)
]

for t in threads: t.start()
for t in threads: t.join()

print(f'Time: {time.time()-start:.2f}s')

# Result: ~1.0 seconds — NO speedup despite 2 threads!
```

Why No Speedup — Detailed Explanation

WHAT WE EXPECTED:

Thread 1 → Core 1 → 10M iterations  done

Thread 2 → Core 2 → 10M iterations  done

Total time ≈ 0.5 seconds (2 cores, half the work each)

WHAT ACTUALLY HAPPENS (GIL at work):

Thread 1 → [GIL] → runs → releases GIL

Thread 2 → [GIL] → runs → releases GIL

Thread 1 → [GIL] → runs → ...

One thread at a time – effectively SEQUENTIAL

Total time ≈ 1.0 seconds – same as single thread!

GIL is released ONLY during I/O waits, not CPU computation.

Adding more threads to CPU tasks = more GIL contention = SLOWER.

■ Section 3 — Multiprocessing Solves It (10_process2.py)

Same Task — Process Instead of Thread

```
from multiprocessing import Process    # Only this line changed!
import time

def cpu_heavy():                        # IDENTICAL function
    print('Crunching numbers...')
    total = 0
    for i in range(10**7):
        total += i
    print('Done!')

# MANDATORY GUARD — always required for multiprocessing
if __name__ == '__main__':
    start = time.time()

    processes = [
        Process(target=cpu_heavy)    # Process, not Thread
        for _ in range(2)
    ]

    for p in processes: p.start()
    for p in processes: p.join()

    print(f'Time: {time.time()-start:.2f}s')
    # Result: ~0.35 seconds — nearly 3x faster!
```

Why Multiprocessing is Faster

Each Process = Completely separate Python interpreter

- Completely separate GIL
- Completely separate memory
- Runs on its OWN CPU core

Process 1: [Own GIL] → Core 1 → 10M iterations freely ■■■■ done
Process 2: [Own GIL] → Core 2 → 10M iterations freely ■■■■ done

|_____|

Both running TRULY in parallel — no waiting!

Threading: ~1.0 seconds (GIL serialises execution)
Multiprocessing: ~0.35 seconds (True parallel — 3x faster!)

The key insight: Changing `threading.Thread` to `Process` and adding the `if __name__ == '__main__':` guard is all it takes to transform a slow CPU-bound threading program into a truly parallel multiprocessing program. The function being executed does not change at all.

■ Section 4 — The if `__name__ == '__main__'` Guard

What Happens Without the Guard

On Windows and macOS, multiprocessing uses the **spawn** start method — a brand-new Python interpreter is launched for each child process, and it **imports the parent script** to find the target function.

If process-creation code is at the top level (not guarded), the child also tries to create processes — which creates grandchildren, which creates more... infinite recursion until the OS runs out of resources.

```
# WITHOUT guard — what happens on Windows/macOS:

from multiprocessing import Process

def work(): pass

p1 = Process(target=work)    # TOP LEVEL — dangerous!
p1.start()
p1.join()

# Child imports this script → sees p1 = Process() at top level
# Child also calls p1.start() → spawns grandchild
# Grandchild imports → spawns great-grandchild → ...
# INFINITE SPAWNING!

# ERROR:
# RuntimeError: An attempt has been made to start a new process
# before the current process has finished its bootstrapping phase.
```

The Correct Pattern

```
from multiprocessing import Process

def work():
    print('Working...')

# CORRECT — all process creation inside the guard
if __name__ == '__main__':
    p1 = Process(target=work)
    p1.start()
    p1.join()

# HOW IT WORKS:
# In the original script: __name__ == '__main__' → block runs
# In a child process:    __name__ == '__mp_main__' → block SKIPPED
```

```
# Child finds the work() function but does NOT spawn more processes  
# Infinite spawning PREVENTED!  
  
# NOTE: threading does NOT need this guard  
# Threads live inside the same process – no re-importing occurs
```


■ Section 5 — Converting Threading Code to Multiprocessing

Side-by-Side Comparison

```
# ■■ THREADING VERSION ████████████████████████████████████████████
```

```
import threading, time
```

```
def cpu_heavy():  
    total = sum(range(10**7))
```

```
threads = [threading.Thread(target=cpu_heavy) for _ in range(2)]  
for t in threads: t.start()  
for t in threads: t.join()
```

```
# Time: ~1.0s – no speedup
```

```
# ■■ MULTIPROCESSING VERSION ████████████████████████████████████████████
```

```
from multiprocessing import Process # Change 1: import  
  
import time
```

```
def cpu_heavy(): # Function unchanged  
    total = sum(range(10**7))
```

```
if __name__ == '__main__': # Change 2: add guard  
    processes = [Process(target=cpu_heavy) for _ in range(2)]  
        # ^--- Change 3: Thread -> Process  
    for p in processes: p.start()  
    for p in processes: p.join()
```

```
# Time: ~0.35s – 3x speedup!
```

What Changes	Threading	Multiprocessing
Import	import threading	from multiprocessing import Process
Class name	threading.Thread(...)	Process(...)
Entry guard	Not required	if __name__ == '__main__': REQUIRED
Memory	Shared — all threads see vars	Separate — no direct sharing
Data sharing	Direct (shared variables)	Queue or Value required
GIL	Shared one GIL per process	Own GIL per process
CPU speedup	No — GIL serialises	Yes — true parallel on cores
Overhead	Low — lightweight	Higher — separate interpreter
Function body	Unchanged	Unchanged

■ Section 6 — Queue: Inter-Process Communication (11_processq.py)

Why Queue is Needed

Each process has its **own private memory space**. A variable modified in Process 1 is completely invisible to Process 2. This is fundamentally different from threading, where all threads share the same memory.

To pass data between processes, Python's multiprocessing module provides **Queue** — a thread and process-safe FIFO data structure that acts as a shared communication channel between processes.

Queue API Reference

Method	Purpose	Behaviour
Queue()	Create a shared queue	Import: from multiprocessing import Queue
q.put(item)	Add item to back of queue	Blocks if queue full (maxsize set)
q.get()	Remove and return front item	Blocks until item is available
q.empty()	Check if queue has no items	Returns True/False — not 100% reliable in MP
q.full()	Check if queue is at capacity	Only meaningful if maxsize was set
q.qsize()	Number of items in queue	Approximate in multiprocessing context
Queue(maxsize=10)	Create bounded queue	put() blocks when queue reaches maxsize

Full Code — 11_processq.py

```
from multiprocessing import Process, Queue

# Function receives queue as argument
def prepare_chai(q):
    result = 'Masala chai is ready!'
    q.put(result)          # Place result in the shared queue
    print('Result placed in queue.')

if __name__ == '__main__':
    q = Queue()            # Create the shared queue

    p = Process(
        target=prepare_chai,
        args=(q,)          # Pass queue as argument (tuple!)
    )
```

```

p.start()

p.join()

result = q.get()      # Retrieve result from queue
print(f'Main got: {result}')

```

Queue with Multiple Processes

```

from multiprocessing import Process, Queue
import time

def worker(q, name, delay):
    time.sleep(delay)
    q.put(f'{name} finished!')    # Each worker puts result

if __name__ == '__main__':
    q = Queue()                  # One queue, shared by all

    processes = [
        Process(target=worker, args=(q, 'Worker 1', 1)),
        Process(target=worker, args=(q, 'Worker 2', 2)),
        Process(target=worker, args=(q, 'Worker 3', 3)),
    ]

    for p in processes: p.start()
    for p in processes: p.join()

    # Collect ALL results from queue
    while not q.empty():
        print(q.get())

# Output:
# Worker 1 finished!
# Worker 2 finished!
# Worker 3 finished!

```

What can be put in a Queue?

Queue uses Python's pickle module to serialise data. Anything that can be pickled can go in a Queue:

- Strings, integers, floats
- Lists and dictionaries
- Custom class instances (if picklable)
- Lambda functions
- File handles
- Database connections
- Unpicklable objects

■ Section 7 — Value: Shared Counter (12_process_value.py)

Value vs Queue — When to Use Which

Feature	Queue	Value
Data type	Any picklable Python object	Single primitive (int, float, bool)
Access pattern	Producer puts, consumer gets	Any process reads/writes .value
Built-in lock	Thread/process-safe by design	Use .get_lock() for safe writes
Use case	Pass results between processes	Shared counter or status flag
Import	from multiprocessing import Queue	from multiprocessing import Value
Read	q.get()	counter.value
Write	q.put(item)	counter.value = x (inside lock)

Full Code — 12_process_value.py

```
from multiprocessing import Process, Value

def increment(counter):
    for _ in range(100_000):          # 100,000 increments
        with counter.get_lock():      # Acquire built-in lock
            counter.value += 1        # Safe modification
                                     # Lock auto-released on exit

if __name__ == '__main__':
    # 'i' = C-type integer, 0 = initial value
    counter = Value('i', 0)

    # 4 processes all sharing the same counter
    processes = [
        Process(target=increment, args=(counter,))
        for _ in range(4)
    ]

    for p in processes: p.start()
    for p in processes: p.join()

    print(f'Final: {counter.value}')

    # 4 processes x 100,000 = 400,000 ■
```

Value Type Codes

Type Code	C Type	Python Type	Example
'b'	signed char	int	Value('b', 0) — small integer
'i'	signed int	int	Value('i', 0) — standard integer
'l'	signed long	int	Value('l', 0) — long integer
'f'	float	float	Value('f', 0.0) — floating point
'd'	double	float	Value('d', 0.0) — double precision
'B'	unsigned char	int	Value('B', 0) — non-negative small int
'I'	unsigned int	int	Value('I', 0) — non-negative integer

Why `counter.get_lock()` is essential:

Even though `Value` stores data in shared memory, the read-modify-write cycle (`counter.value += 1`) is not atomic — it is three operations. Without the lock, two processes can both read the same value, both add 1, and both write the same result — losing one increment.

`Value`'s built-in lock (`counter.get_lock()`) prevents this. It is equivalent to `threading.Lock()` but designed specifically for use with `Value` objects across process boundaries.

■ Section 8 — Threading vs Multiprocessing Speed Comparison

Benchmark Results

Task Size	Sequential	Threading (2)	Multiprocessing (2)	MP Speedup
10 ⁶	~0.1s	~0.1s	~0.1s	~1x (overhead dominates)
10 ⁷	~1.0s	~1.0s	~0.35s	~3x faster
10 ⁸	~10s	~10s	~3.5s	~3x faster
10 ⁹	~100s	~100s	~35s	~3x faster

Important observation: For very small tasks (10⁶), multiprocessing may not be faster because the overhead of spawning separate processes (starting a new Python interpreter, allocating separate memory) exceeds the time saved by parallelism. Multiprocessing is most beneficial when each task takes at least a few hundred milliseconds of CPU time.

■ Section 9 — Real World Use Cases

Where Multiprocessing is Used in Production

```
# 1. BATCH IMAGE PROCESSING  
# ████████████████████████████████████████████████████████████████████████████  
  
from multiprocessing import Pool  
from PIL import Image  
  
def apply_filter(image_path):  
    img = Image.open(image_path)  
    # Apply some CPU-intensive filter  
    img = img.convert('L') # Grayscale  
    img.save(image_path.replace('.jpg', '_processed.jpg'))  
    return f'Done: {image_path}'  
  
if __name__ == '__main__':  
    image_paths = ['img1.jpg', 'img2.jpg', 'img3.jpg', ...]  
  
    with Pool(4) as pool:          # 4 worker processes  
        results = pool.map(apply_filter, image_paths)  
  
    # Each image processed on a separate core – 4x speedup!
```

```
# 2. ML DATA LOADING (PyTorch style)
# ████████████████████████████████████████████████████████████████████████████

from torch.utils.data import DataLoader

loader = DataLoader(
    dataset,
    batch_size=32,
    num_workers=4,      # 4 processes loading data in parallel
)

# PyTorch internally uses multiprocessing for data prefetching


# 3. LARGE CSV DATA PROCESSING
# ████████████████████████████████████████████████████████████████████████████

from multiprocessing import Pool

import pandas as pd

def process_chunk(chunk_path):
    df = pd.read_csv(chunk_path)

    # Heavy transformation

    return df.describe()
```

```

if __name__ == '__main__':
    chunks = ['chunk1.csv', 'chunk2.csv', 'chunk3.csv']
    with Pool(4) as pool:
        summaries = pool.map(process_chunk, chunks)

```

Domain	Task	Why MP?	Tool
Computer Vision	Batch image processing	CPU-bound transforms	multiprocessing.Pool
Machine Learning	Data loading/augmentation	CPU-bound preprocessing	torch DataLoader (MP)
Data Engineering	Large CSV transformation	CPU-bound parsing	multiprocessing.Pool
Scientific Computing	Numerical simulations	Pure CPU computation	multiprocessing.Process
Financial	Risk model calculations	Heavy math	multiprocessing.Pool
Video Processing	Frame extraction/encoding	CPU-bound encoding	multiprocessing.Process
Web Scraping	Download (threading) + parse MP	Parsing is CPU-heavy	threading + MP combo

■ Section 10 — Interview Questions & Model Answers

Q1. Why doesn't threading speed up CPU-bound tasks in Python?

Due to the Global Interpreter Lock (GIL) in CPython. During CPU computation, a thread must hold the GIL to execute Python bytecode and modify Python objects. The GIL is only released during I/O waits, not during computation. With two CPU-bound threads, they constantly compete for the single GIL — effectively executing sequentially on one core regardless of how many cores are available. Adding more threads to a CPU task may even slow it down due to GIL contention overhead.

Q2. How does multiprocessing bypass the GIL?

Each Process spawned by the multiprocessing module is a completely independent OS process with its own Python interpreter, its own memory space, and its own GIL. Since there is no shared GIL between processes, each process can execute Python bytecode on its own CPU core simultaneously — achieving true parallelism. The trade-off is that processes cannot share memory directly and have higher startup overhead compared to threads.

Q3. Why must multiprocessing code use `if __name__ == '__main__':`?

On Windows and macOS, Python's multiprocessing uses the 'spawn' start method — a fresh Python interpreter is launched for each child process, and it imports the parent script to locate the target function. Without the guard, the import would re-execute the process-creation code in the child, spawning more children in an infinite loop until the OS runs out of resources. The guard ensures process-creation code only runs in the original (main) script. Threading does not need this guard because threads do not re-import the script.

Q4. How do processes communicate with each other?

Processes have separate memory and cannot share variables directly. Python's multiprocessing module provides two primary IPC mechanisms: Queue — a process-safe FIFO queue where one process puts data and another gets it. Suitable for passing any pickleable Python object between processes. Value — a shared memory primitive for simple numeric types (int, float). It comes with a built-in lock accessed via `counter.get_lock()`. Other options include Pipe (bidirectional channel between two processes) and shared Array for bulk numeric data.

Q5. What is multiprocessing.Value and why must you use `get_lock()`?

Value creates a shared memory object that all processes can read and write. You specify a C type code ('i' for int, 'f' for float) and an initial value: `counter = Value('i', 0)`. Access the number via `counter.value`. However, `counter.value += 1` is not atomic — it is a read-modify-write sequence. Without a lock, two processes can both read the same value and both write the same result, losing increments. `counter.get_lock()` returns Value's built-in mutex — using `'with counter.get_lock():'` ensures only one process modifies the value at a time.

Q6. What is the difference between Queue and Value for IPC?

Queue is suitable for passing complex, variable data between processes — strings, lists, dictionaries, custom objects (anything picklable). It works like a shared post box: one process puts items in, another gets them out. Value is designed for a single shared primitive number (int or float). It is simpler and faster for a shared counter or status flag, and comes with a built-in lock. Use Queue when you need to pass structured results from worker processes back to the main process. Use Value when you need a single shared numeric state visible to all processes simultaneously.

Q7. When would you combine threading and multiprocessing?

A common real-world pattern is to use threading for I/O work and multiprocessing for CPU work within the same application. For example, in a web scraper: threading handles downloading many pages concurrently (I/O-bound — GIL released). Once pages are downloaded, multiprocessing handles parsing and processing the HTML (CPU-bound — needs true parallelism). Another example: a machine learning pipeline where threading prefetches data from disk while multiprocessing processes/augments the data on multiple cores.

Q8. What are the drawbacks of multiprocessing compared to threading?

Higher startup overhead: spawning a process creates a new Python interpreter and allocates separate memory — much slower than creating a thread. No shared memory: processes cannot directly share variables, requiring IPC mechanisms (Queue, Value, Pipe) which add complexity and serialisation overhead. Higher memory usage: each process has its own copy of the program's data in memory. Complexity: coordinating results from multiple processes is more involved than reading shared variables from threads. For small tasks, the overhead of process creation may exceed the time saved — multiprocessing is only beneficial when each task takes at least hundreds of milliseconds of CPU-intensive work.

■ Section 11 — Quick Revision Cheatsheet

Key One-Liner Definitions

```
Process          = Independent program with own memory, interpreter, and GIL
multiprocessing  = Python module for true CPU parallelism
Process()        = Creates a process – NOT started until .start()
.start()         = Spawn process on its own CPU core
.join()          = Block caller until process terminates
if __name__      = Entry guard – prevents infinite process spawning
Queue()          = Process-safe FIFO channel for complex data
q.put(item)       = Place data into the queue
q.get()           = Retrieve data from the queue (blocks until available)
Value('i', 0)    = Shared integer across processes
counter.value     = Read or write the shared number
counter.get_lock() = Built-in mutex for safe Value modification
Pool(n)          = Manage a pool of n worker processes (map/apply)
IPC              = Inter-Process Communication – Queue, Value, Pipe
```

Threading vs Multiprocessing — Final Decision Table

Criterion	Use Threading	Use Multiprocessing
Task type	I/O bound (wait-heavy)	CPU bound (compute-heavy)
Memory	Shared — direct access	Separate — Queue/Value needed
GIL impact	Released on I/O — fast	Own GIL per process — fast
Startup cost	Low	High (new interpreter)
Speedup potential	High for I/O tasks	High for CPU tasks
Example tasks	DB queries, API calls, scraping	ML, image proc, big math
if __name__ guard	Not needed	MANDATORY
Python module	threading	multiprocessing
Data sharing	Shared variables (use Lock)	Queue, Value, Pipe

Common Mistakes to Avoid

MISTAKE 1: Using threading for heavy CPU computation

FIX : Use multiprocessing — each process gets its own GIL

MISTAKE 2: Forgetting `if __name__ == '__main__':` in multiprocessing
FIX : Always wrap process creation in this guard

MISTAKE 3: `args=('value')` – single-item without trailing comma
FIX : `args=('value',)` – trailing comma makes it a tuple

MISTAKE 4: Modifying `Value` without `get_lock()`
FIX : Always: with `counter.get_lock(): counter.value += 1`

MISTAKE 5: Trying to read a `Queue` after all processes joined
without checking `empty` – may block forever on `get()`
FIX : Use `while not q.empty(): q.get()` or `q.get(timeout=1)`

MISTAKE 6: Using multiprocessing for tiny tasks
FIX : Process spawn overhead negates speedup for `< ~100ms` tasks
Use `threading` or `asyncio` for lightweight concurrent work

Golden Rules for Multiprocessing Interviews:

1. "CPU-bound task? Use multiprocessing — each process has its own GIL."
2. "Processes cannot share memory — use `Queue` for complex data, `Value` for numbers."
3. "Always add `if __name__ == '__main__':` — without it, infinite spawning on Windows."
4. "Always use `counter.get_lock()` when modifying a `Value` — it is not thread/process safe alone."